TinyIDS: Lightweight, explainable intrusion detection for IoT gateways

Background and motivation

IoT gateways are the natural place to stop attacks, but they lack the compute to run heavy intrusion detection models. Cheap devices such as cameras, plugs and sensors ship with weak security and are routinely recruited into botnets (Mirai-style DDoS, brute force, reconnaissance).

Most published IoT intrusion detection systems (IDS) report high accuracy on a workstation and stop there. Few measure whether the model fits a real gateway's latency, memory and energy budget, and few explain why an alert fired. This project closes that gap on a Raspberry Pi gateway using the public CICIoT2023 dataset.

Problem statement and research questions

Design an intrusion detector that runs in real time on a Raspberry Pi 4 gateway, uses a small set of traffic features, and gives a human-readable reason for every alert.

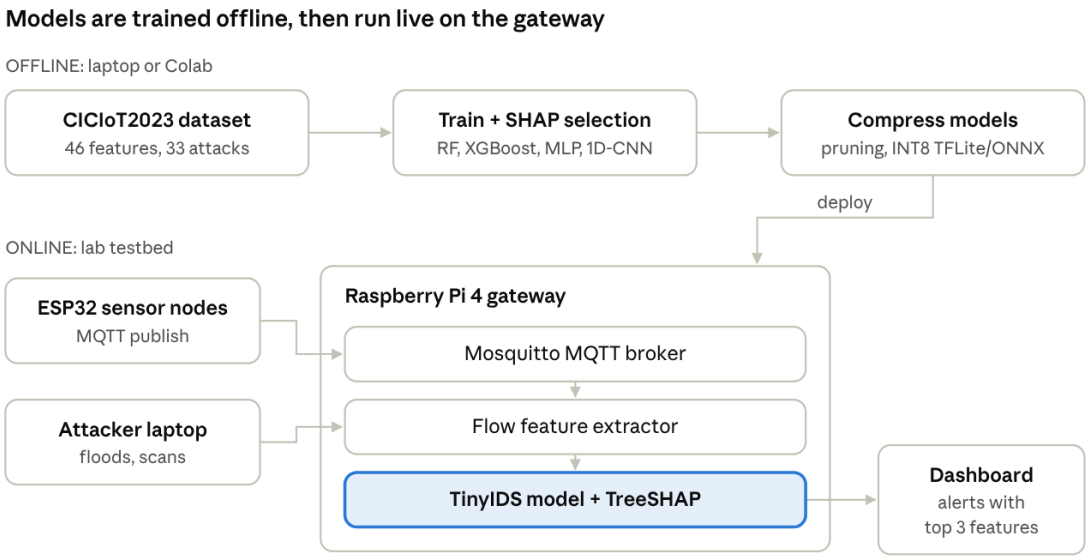
1. **RQ1:** How few flow features (out of 46 in CICIoT2023) are needed to keep macro-F1 within 2% of the full-feature model?
2. **RQ2:** Which compressed model (quantized tree ensemble, pruned MLP, small 1D-CNN) gives the best accuracy per millisecond and per megabyte on the Pi?
3. **RQ3:** Can SHAP-based explanations be produced on the gateway fast enough to accompany live alerts?

Objectives

1. Benchmark at least four classifiers on CICIoT2023 for binary (benign vs attack), 8-class and 34-class detection.
2. Reduce the feature set with SHAP-based selection and measure the accuracy trade-off.
3. Compress the best models (pruning, INT8 quantization) and deploy them on a Raspberry Pi 4.
4. Build a small MQTT testbed with ESP32 nodes and detect replayed or live attacks on it.
5. Provide a dashboard that shows each alert with its top contributing features.
6. Write up the results as a 6-page conference paper.

Methodology

Models are trained offline on CICIoT2023, shrunk, then run on the Pi against live testbed traffic.



TinyIDS system architecture · offline training, online gateway

The compressed model is the only artifact carried from the offline pipeline to the gateway; everything in the online row runs on the Pi in real time.

1. Data preparation. Use the CICIoT2023 CSV files (33 attacks, 105 devices, 46 flow features). Start from a stratified subset (for example 5–10% of rows) or the preprocessed Mendeley subsets to keep training manageable. Handle class imbalance with class weights or undersampling of the DDoS classes, and scale features with a scaler fitted on training data only.

2. Baseline models. Train Random Forest, XGBoost/LightGBM, a 3-layer MLP and a small 1D-CNN. Report accuracy, macro-F1, per-class recall and confusion matrices for the 2-, 8- and 34-class tasks, using a held-out test split.

3. Feature selection. Rank features by mean absolute SHAP value from the best tree model. Retrain with the top 5, 10, 15 and 20 features and plot macro-F1 against feature count (answers RQ1).

4. Compression and deployment. Prune tree depth and estimator count; convert the MLP/CNN to TensorFlow Lite or ONNX with INT8 quantization. On the Pi, measure model size (KB), peak RAM (MB), latency per flow (ms), throughput (flows/s) and energy per 1,000 inferences with a USB power meter (answers RQ2).

5. Testbed. Two or three ESP32 nodes publish sensor data over MQTT to a Mosquitto broker on the Pi. An attacker laptop generates SYN/UDP floods (hping3), port scans (nmap) and MQTT brute force. The Pi extracts flow features with CICFlowMeter or a lightweight scapy-based extractor and classifies each flow.

6. Explainability. Compute TreeSHAP for each flagged flow, show the top three features in the alert, and measure the extra latency (answers RQ3).

Validity note: features computed by a different extractor on testbed traffic may not match CICIoT2023's exactly. Report testbed results separately and treat them as a demonstration rather than the main accuracy figure.

Hardware:

Item	Qty	Purpose
Raspberry Pi 4 (4 GB) with power supply, 32 GB microSD, case	1	Gateway: broker, feature extraction, IDS
ESP32 dev board + DHT22 sensor	2–3	MQTT sensor nodes
USB power meter (inline)	1	Energy measurement
Laptop	1	Model training and attacker machine
Wi-Fi router or Pi hotspot (existing)	1	Isolated test network

Software: Python 3, scikit-learn, XGBoost/LightGBM, TensorFlow/TFLite or ONNX Runtime, SHAP, pandas; Mosquitto MQTT broker; CICFlowMeter or scapy; hping3 and nmap for attack generation; Grafana + InfluxDB or Streamlit for the dashboard; Arduino IDE or PlatformIO for the ESP32s. A free Google Colab or Kaggle GPU is enough for training.

Four-week timeline

Week	Work	Checkpoint at end of week
1	Literature review; download and explore CICIoT2023; clean, sample and balance data; order hardware	Survey table; cleaned dataset; hardware in hand
2	Train RF, XGBoost, MLP, 1D-CNN on 2/8/34-class tasks; SHAP feature ranking; retrain on top-k features	Results tables; F1 vs feature-count plot
3	Compress and deploy models on the Pi; benchmark latency, RAM, energy; set up ESP32 + MQTT testbed; run attacks and live detection	Benchmark table; working live demo
4	Add SHAP explanations to alerts; build dashboard; record demo; write paper and report	Paper draft, report, demo video, GitHub repo

Deliverables and evaluation metrics

Deliverables

- ☐Public GitHub repository: preprocessing, training, deployment code, trained models, README
- ☐Raspberry Pi gateway prototype detecting attacks live on the testbed
- ☐Benchmark table: accuracy, macro-F1, model size, RAM, latency, energy per model
- ☐Alert dashboard showing each alert with its top 3 SHAP features
- ☐6-page paper in IEEE conference format
- ☐Project report (25–35 pages) and 3-minute demo video

Evaluation metrics

Category	Metric	Target
Detection	Macro-F1, per-class recall, false-positive rate	Binary F1 ≥ 0.98; 8-class within 2% of full-feature model
Efficiency	Model size, peak RAM on Pi	< 1 MB model; < 100 MB RAM
Speed	Latency per flow, throughput	< 5 ms per flow; ≥ 500 flows/s
Energy	mJ per inference (USB meter, idle subtracted)	Reported for every model
Explainability	Extra latency of SHAP per alert	< 50 ms

References

1. E. C. P. Neto et al., "CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment," *Sensors*, vol. 23, no. 13, 5941, 2023. DOI: 10.3390/s23135941.
2. Canadian Institute for Cybersecurity, "CIC IoT Dataset 2023," University of New Brunswick (dataset download).
3. "Realguard: A Lightweight Network Intrusion Detection System for IoT Gateways," *Sensors*, 2022. Closest baseline for this project.
4. "Think Fast: Real-Time IoT Intrusion Reasoning Using IDS and LLMs at the Edge Gateway," arXiv:2511.18230. Recent work for related-work and future-work sections.
5. "Derived CICIoT 2023 datasets," Mendeley Data (preprocessed subsets).
6. S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," *NeurIPS*, 2017 (SHAP).